Ordenamiento por Intercambio (Bubble Sort), del juego de tarjetas al código

Objetivo

Comprender el algoritmo de **Bubble Sort** mediante la simulación con tarjetas. Después, diseñar una solución en **pseudocódigo** e implementarla en **Java**.

Materiales

- Tarjetas con números o nombres (mínimo 5). Ejemplo: 34, 12, 25, 8, 19.
- Hojas para registrar las pasadas.
- Computadora con JDK instalado y Bloc de Notas

Fase 1: Simulación con tarjetas (se conserva de la actividad original)

1. Formen equipos de 5 o más personas. Cada integrante es una tarjeta y se colocan en fila.
2. Comparen pares adyacentes de izquierda a derecha. Si la tarjeta de la izquierda es mayor que la de la derecha, intercambian lugares.
3. Recorrer toda la fila es **una pasada**. Repitan las pasadas hasta que ya no haya intercambios.
4. **Registren en una tabla** el estado de la fila al terminar cada pasada, el número de comparaciones y el número de intercambios.

Fase 2: Propuesta de solución en pseudocódigo (en binas)

A partir del registro de la Fase 1, escriban su propio pseudocódigo. Debe responder a estas preguntas:

- ¿Qué estructura de datos representa la fila de tarjetas?

Un arreglo de enteros cada posición es el lugar de una persona en la fila.

- ¿Qué ciclo representa las **pasadas** y cuál representa las **comparaciones** dentro de una pasada?

El ciclo pasado o externo representa cada recorrido completo de la fila. El ciclo interno compara cada par de tarjetas vecinas dentro de esa pasada.

- ¿Cómo se realiza el **intercambio** de dos elementos sin perder uno de ellos?

Con una variable auxiliar.

- ¿Cómo sabe el algoritmo que **ya terminó**, es decir, cuándo detenerse antes de completar todas las pasadas?

Cuando ya hizo una pasada completa sin intercambios, y eso significa que ya no hay nada que acomodar por que ya está ordenado.

- ¿Cuántos elementos hay que comparar en cada pasada? Piensen en qué tarjetas ya quedaron en su lugar definitivo.

En la pasada k se hacen n − k comparaciones, porque las últimas k − 1 tarjetas ya están en su lugar. Con 5 tarjetas: 4, 3, 2 y 1.

Entregable: el pseudocódigo en hoja o archivo, con nombres de variables claros y sangría.

```
// Pseudocódigo del ordenamiento por intercambio (Bubble Sort)
// Autora: Noemi Mata Ruiz

Algoritmo BubbleSort
    Definir fila, i, j, aux, comparaciones, intercambios Como Entero
    Definir huboCambio Como Logico
    Dimension fila[5]

    // Primero pido el número de cada compañero, así como nos formamos en la fila
    Para i <- 1 Hasta 5 Hacer
        Escribir "Numero del compañero ", i, ":"
        Leer fila[i]
    FinPara

    // Pongo los contadores en cero para ir contando lo que hacemos
    comparaciones <- 0
    intercambios <- 0
    huboCambio <- Verdadero
    i <- 1

    // Aquí empiezan las pasadas, las repito mientras alguien se siga cambiando de lugar
    Mientras i < 5 Y huboCambio Hacer
        huboCambio <- Falso

        // En cada pasada voy comparando a cada compañero con el que tiene a su derecha
        // Uso 5 - i porque los últimos ya están en su lugar y ya no los tomo en cuenta
        Para j <- 1 Hasta 5 - i Hacer
            comparaciones <- comparaciones + 1

            // Si el de la izquierda es mayor, se cambian de lugar
            Si fila[j] > fila[j + 1] Entonces
                // Uso aux para guardar un número y no perderlo al hacer el cambio
                aux <- fila[j]
                fila[j] <- fila[j + 1]
```

```
            fila[j + 1] <- aux
            intercambios <- intercambios + 1
            // Marco que sí hubo un cambio en esta pasada
            huboCambio <- Verdadero
        FinSi
    FinPara

    // Al terminar la pasada muestro cómo quedó la fila
    Escribir Sin Saltar "Pasada ", i, ": "
    Para j <- 1 Hasta 5 Hacer
        Escribir Sin Saltar fila[j], " "
    FinPara
    Escribir ""

    i <- i + 1
  FinMientras

  // Cuando ya nadie se cambió de lugar, la fila quedó ordenada y muestro los resultados
  Escribir "Pasadas: ", i - 1
  Escribir "Comparaciones: ", comparaciones
  Escribir "Intercambios: ", intercambios
FinAlgoritmo
```

Fase 3: Implementación en Java

1. Traduzcan su pseudocódigo a un programa en Java con un método bubbleSort(int[] arreglo).
2. El programa debe mostrar el arreglo **después de cada pasada**, para compararlo con el registro de la Fase 1.
3. Debe contar e imprimir el **total de comparaciones, intercambios y pasadas**.
4. Prueben con el arreglo {34, 12, 25, 8, 19}, con uno ya ordenado y con uno en orden inverso.
5. Variación opcional: ordenar un arreglo de String alfabéticamente con compareTo.

Fase 4: Reflexión

- ¿Coincide la salida del programa con lo que registraron con las tarjetas?
  Sí. La fila quedó igual en cada pasada y los totales coincidieron: 4 pasadas, 10 comparaciones y 7 intercambios.

- ¿Qué cambios tuvo que hacer su pseudocódigo al pasarlo a Java?
  Los índices empiezan en 0 y no en 1. Mientras, Para y Si se volvieron while, for e if. Usé Scanner para leer los números y separé el código en clases con encapsulamiento.

- ¿Qué pasa cuando el arreglo ya está ordenado? ¿Sigue comparando?
  No. Hace una sola pasada con 4 comparaciones y se detiene, porque no hubo ningún intercambio.

- ¿Es eficiente con listas muy grandes? Relacionen su respuesta con el número de comparaciones ($O(n^2)$).
  No. Es $O(n^2)$: con 5 datos hace 10 comparaciones, pero con 1,000 hace 499,500. Para listas grandes conviene usar Merge Sort o Quick Sort.

Lista de cotejo (5 puntos)

#	Criterio	Cumple	No cumple	Observaciones
1	**Simulación y registro:** la tabla de pasadas con tarjetas es completa y correcta (estado de la fila, comparaciones e intercambios por pasada).			
2	**Pseudocódigo:** es claro, está estructurado y contiene los dos ciclos (pasadas y comparaciones), la condición de comparación, el intercambio con variable auxiliar y la condición de paro.			
3	**Implementación en Java:** el programa compila, ejecuta y ordena correctamente el arreglo de ejemplo y los casos de prueba (ordenado e inverso).			
4	**Coherencia pseudocódigo–código:** el programa respeta la lógica del pseudocódigo propuesto y muestra el arreglo tras cada pasada, junto con los contadores.			
5	**Reflexión y buenas prácticas:** responde las preguntas de reflexión con argumentos, incluida la eficiencia, y el código usa nombres significativos, sangría y comentarios.			